The k-Partition Problem

Anastasiia Artemova Maksym Suprun Pramika Singh Jha Dmitry Hais

October 2026

Contents

1 Problem Statement **2**

2 Proposed Solutions **2**
2.1 Common Greedy Initialization 2
2.2 Solution 1: Energy-Based Subset-Exchange Local Search 3
2.2.1 Energy Function 3
2.2.2 Candidate Exchanges for Energy Change 3
2.2.3 Pseudocode 4
2.2.4 Time Complexity 4
2.2.5 Termination and Local Optimality 5
2.2.6 Limitations 5
2.3 Solution 2: Polynomial-Time Single-Element Swap Local Search 6
2.3.1 Algorithm 6
2.3.2 Pseudocode 7
2.3.3 Polynomial Time Complexity 7
2.3.4 Termination and Limitations 8

3 Comparison of the Two Approaches **8**

4 Implementation and Input/Output **8**

5 Compilation and Execution **8**
5.1 Compilation 8
5.2 Execution 9
5.3 Input Format 9
5.4 Output Format 9

6 Conclusion **10**

1 Problem Statement

Given a collection of n positive integers $A = (a_1, \ldots, a_n)$ and an integer $1 \leq k \leq n$, partition the elements into k nonempty, pairwise disjoint groups $G_1, \ldots, G_k$. Every input occurrence must belong to exactly one group. If $S_i = \sum_{x \in G_i} x$, the objective is

$$D(G_1, \ldots, G_k) = \min \left(\max_{1 \leq i \leq k} S_i - \min_{1 \leq i \leq k} S_i \right) \tag{1}$$

The ideal value is $D = 0$, although an equal-sum partition need not exist.

2 Proposed Solutions

For the given problem we designed 2 solutions:

- Energy-Based Subset-Exchange Local Search
- Polynomial-Time Single-Element Swap Local Search

2.1 Common Greedy Initialization

Both proposed methods begin with the same greedy construction. Create k empty groups with zero sums, sort the input in non-increasing order, and assign each element to a group with the currently smallest sum (ties may be broken by group index).

Sorting costs $O(n \log n)$. With a linear scan over the k group sums for each of the n elements, assignment costs $O(nk)$. Hence the greedy initialization costs

$$T_{\text{greedy}}(n, k) = O(n \log n + nk). \tag{2}$$

Algorithm 1 Greedy initialization

Require: Positive integers A, number of groups k
Ensure: Groups $G_1, \ldots, G_k$ and their sums $S_1, \ldots, S_k$

```
Initialize all G_i ← ∅ and S_i ← 0
Sort A in non-increasing order
for each x ∈ A do
    i ← arg min_{1≤j≤k} S_j
    Insert x into G_i; S_i ← S_i + x
end for
return (G, S)
```

2.2 Solution 1: Energy-Based Subset-Exchange Local Search

2.2.1 Energy Function

Let $T = \sum_{j=1}^{n} a_j$ and $\mu = T/k$. The equal-sum target is $S_i = \mu$ for every i. There are some cases when an equal-sum integer partition is impossible. For example, with $A = (1, 2, 5)$ and $k = 3$, $\mu = 8/3$. Even when μ is an integer, indivisible input elements can prevent equality.

We introduce the squared-deviation measure

$$V = \sum_{i=1}^{k} (S_i - \mu)^2. \tag{3}$$

Expanding and using $\sum_i S_i = T = k\mu$ yields

$$V = \sum_{i=1}^{k} S_i^2 - 2\mu T + k\mu^2 \tag{4}$$

$$= \sum_{i=1}^{k} S_i^2 - \frac{T^2}{k}. \tag{5}$$

Since T and k are fixed, minimizing V is equivalent to minimizing the simpler energy

$$E = \sum_{i=1}^{k} S_i^2. \tag{6}$$

Energy measures overall imbalance but is not identical to the objective D in Equation (1).

2.2.2 Candidate Exchanges for Energy Change

For each unordered pair of groups (G_a, G_b), enumerate a subset $A' \subseteq G_a$ and a subset $B' \subseteq G_b$. Empty selections are allowed, enabling one-way transfers, but exchanges that leave either resulting group empty are excluded.

Define the signed net transfer

$$d = \sum_{x \in A'} x - \sum_{y \in B'} y. \tag{7}$$

The new group sums are $S'_a = S_a - d$ and $S'_b = S_b + d$. Since other group sums do not change,

$$\Delta E = (S_a - d)^2 + (S_b + d)^2 - S_a^2 - S_b^2 \tag{8}$$

$$= -2S_a d + 2S_b d + 2d^2 \tag{9}$$

$$= \boxed{2d(d - (S_a - S_b))}. \tag{10}$$

Once the subset sums are available, a candidate's energy change can be evaluated in $O(1)$ arithmetic operations. The algorithm chooses the candidate with the most negative ΔE over all pairs, applies it, and repeats. It stops when no candidate has $\Delta E < 0$ (or when a zero-difference solution has been found).

Because energy and the original objective are different, the implementation separately retains the best *visited* partition according to the pair (D, E): lower D takes priority, and lower E breaks ties.

2.2.3 Pseudocode

Algorithm 2 Energy-based subset-exchange search

```
(G, S) ← GreedyInitialization(A, k)
best ← G; D_best ← D(G); E_best ← E(G)
while D_best > 0 do
    δ_best ← 0; move ← none
    for each pair 1 ≤ a < b ≤ k do
        Enumerate subsets of G_a and G_b, storing their sums
        for each pair of subsets (A', B') do
            if either resulting group would be empty then
                continue
            end if
            d ← Σ A' − Σ B'
            δ ← 2d(d − (S_a − S_b))
            if δ < δ_best then
                δ_best ← δ; move ← (a, b, A', B')
            end if
        end for
    end for
    if move = none then
        break
    end if
    Apply move and update S
    if D(G) < D_best or (D(G) = D_best and E(G) < E_best) then
        best ← G; D_best ← D(G); E_best ← E(G)
    end if
end while
return best
```

2.2.4 Time Complexity

Let $m_i = |G_i|$, where $\sum_{i=1}^{k} m_i = n$. A group with m_i elements has 2^{m_i} subsets. Generating and storing their index lists requires $O(m_i 2^{m_i})$ time in the implementation under consideration. For groups a and b, the number of candidate exchanges is

$$2^{m_a} 2^{m_b} = 2^{m_a + m_b}. \tag{11}$$

Although the energy evaluation itself is constant time, recording a best candidate can copy up to $m_a + m_b$ indices. Therefore:

$$O\big((m_a + m_b) 2^{m_a + m_b}\big). \tag{12}$$

Across all pairs, a complete local-search pass costs at most

$$O\left(\sum_{1 \le a < b \le k} (m_a + m_b) 2^{m_a + m_b} \right). \tag{13}$$

For approximately equal group cardinalities $m_i \approx n/k$, Equation (13) gives the balanced-cardinality estimate

$$O\left(k^2 \frac{n}{k} 2^{2n/k} \right) = O\left(kn 2^{2n/k} \right). \tag{14}$$

With I local-search passes, the estimated total cost, including greedy initialization, is

$$\boxed{O\left(n \log n + nk + Ikn2^{2n/k}\right)}. \tag{15}$$

This is not a worst-case polynomial bound: group cardinalities need not be balanced, and I has no polynomial bound imposed by this algorithm. A worst-case per-pass bound is $O(kn2^n)$. The proposed energy-based local search's practical computational cost depends strongly on the number of subsets k and the distribution of elements among them. To illustrate the effect of increasing k, consider $n = 24$ elements and $I = 5$ iterations. We compare the estimated number of candidate evaluations against $2^n = 16\,777\,216$.

Table 1: Estimated candidate evaluations for different values of k ($n = 24$, $I = 5$).

k	$Ik^2 2^{2n/k}$	2^n	Comparison
2	335,544,320	16,777,216	20× worse
3	2,949,120	16,777,216	5.7× better
4	327,680	16,777,216	51× better
6	46,080	16,777,216	364× better
8	20,480	16,777,216	819× better

As shown in Table 1, increasing the number of subsets significantly reduces the estimated number of candidate evaluations. This behavior results from the exponential term $2^{2n/k}$, which decreases rapidly as k grows.

2.2.5 Termination and Local Optimality

Every accepted move has $\Delta E < 0$, so the energy strictly decreases. Since there are finitely many assignments of n input occurrences to k groups, the search cannot continue indefinitely. It terminates after finitely many accepted moves.

If the search stops because no improving exchange exists, its current search state is a local minimum of E relative to valid exchanges between two groups. If it stops early because a perfect partition has been found, that partition is globally optimal for D. The separately retained best-by-(D, E) partition need not itself be a local minimum of E, although it is no worse than the greedy partition with respect to D.

2.2.6 Limitations

Consider

$$A = (25, 20, 18, 15, 12, 10, 8, 7, 5), \qquad k = 4, \qquad T = 120.$$

The ideal sum is $T/k = 30$. Greedy initialization produces:

Group	Elements	Sum
G_1	$25, 7$	32
G_2	$20, 8$	28
G_3	$18, 10$	28
G_4	$15, 12, 5$	32

Here $D = 4$ and $E = 32^2 + 28^2 + 28^2 + 32^2 = 3616$.

In the first iteration, exchanging 18 from G_3 with $(15, 5)$ from G_4 gives $d = 18 - (15 + 5) = -2$ and

$$\Delta E = 2(-2)(-2 - (28 - 32)) = -8.$$

The updated partition is:

Group	Elements	Sum
G_1	$25, 7$	32
G_2	$20, 8$	28
G_3	$10, 15, 5$	30
G_4	$12, 18$	30

Now $E = 3608$, but $D = 4$ remains unchanged. The observed implementation stops after this iteration because it finds no further strictly energy-decreasing pairwise exchange.

Nevertheless, a globally optimal partition exists:

Group	Elements	Sum
G_1	$25, 5$	30
G_2	$20, 10$	30
G_3	$15, 8, 7$	30
G_4	$18, 12$	30

It achieves $D = 0$ and $E = 4 \cdot 30^2 = 3600$. The example shows that strict pairwise energy descent can get stuck despite the existence of a better global solution. Reaching it may require coordinated moves across more than two groups or moves that are not immediately energy-decreasing.

2.3 Solution 2: Polynomial-Time Single-Element Swap Local Search

2.3.1 Algorithm

The second approach uses the same greedy initialization, but replaces exponential subset enumeration with single-element swaps. It considers every pair of groups (G_a, G_b) having unequal current sums, and tries swapping one element $x \in G_a$ with one element $y \in G_b$.

The group sums after the trial swap are

$$S'_a = S_a - x + y, \qquad S'_b = S_b - y + x. \tag{16}$$

The algorithm recomputes the global difference D' across all k groups. It accepts the swap immediately if $D' < D_{\text{current}}$; otherwise, it undoes the trial swap. A complete pass continues scanning candidate swaps, possibly accepting multiple improving swaps. The next pass starts only if at least one improvement was made.

The local search terminates when $D = 0$, no swap improves D during a complete pass, or the number of passes reaches

$$I_{\max} = n^2. \tag{17}$$

The iteration cap guarantees polynomial running time. Unlike Solution 1, this method directly optimizes the original objective D, but its neighborhood contains only one-for-one swaps; it cannot perform one-way transfers or multi-element exchanges.

2.3.2 Pseudocode

Algorithm 3 Polynomial single-element swap search

```
(G, S) ← GreedyInitialization(A, k)
pass ← 0; I_max ← n^2
while pass < I_max do
    D_current ← max(S) − min(S)
    if D_current = 0 then
        break
    end if
    improved ← false
    for each pair 1 ≤ a < b ≤ k do
        if S_a = S_b then
            continue
        end if
        for each element x ∈ G_a do
            for each element y ∈ G_b do
                Tentatively swap x and y and update S_a, S_b
                D' ← max(S) − min(S)
                if D' < D_current then
                    D_current ← D'; improved ← true
                else
                    Undo the swap and restore S_a, S_b
                end if
            end for
        end for
    end for
    if improved = false then
        break
    end if
    pass ← pass + 1
end while
return G
```

2.3.3 Polynomial Time Complexity

Across all unordered pairs of groups, the number of possible element pairs is

$$\sum_{1\le a<b\le k} m_a m_b = \frac{1}{2}\left(n^2 - \sum_{i=1}^{k} m_i^2\right) \le \frac{n^2}{2}. \tag{18}$$

Each trial swap updates two sums in $O(1)$ time and recomputes the global minimum and maximum over k sums in $O(k)$ time. Thus, one complete pass costs

$$O(n^2 k). \tag{19}$$

With at most n^2 passes, the total running time is

$$T_{\text{poly}}(n,k) = O(n\log n + nk + n^2 \cdot n^2 k) \tag{20}$$

$$= \boxed{O(n\log n + n^4 k)}. \tag{21}$$

Because $k \le n$, this is bounded by $O(n^5)$.

2.3.4 Termination and Limitations

Every accepted swap strictly reduces the integer-valued objective D, so accepted swaps cannot cycle. The explicit n^2 pass limit also guarantees termination even without relying on the magnitude of the input values.

If the algorithm stops because an entire pass contains no improving swap, it is locally optimal with respect to its tested one-for-one swaps. If it stops because the iteration limit is reached, such local optimality is not guaranteed. Neither condition guarantees a globally optimal partition.

The method can miss improvements that require moving one element without a compensating exchange, swapping multiple elements, or temporarily accepting a non-improving move. Furthermore, immediate acceptance means that the final partition can depend on the order in which candidate swaps are examined.

3 Comparison of the Two Approaches

Property	Energy-based subset exchanges	Polynomial single-element swaps
Initialization	Descending greedy assignment	Descending greedy assignment
Objective used for moves	Minimize $E = \sum_i S_i^2$	Minimize $D = \max_i S_i - \min_i S_i$
Candidate neighborhood	All valid subset exchanges between any two groups	One element exchanged between any two groups
Acceptance	Best strictly energy-decreasing candidate per pass	Immediately accept any strictly D-decreasing candidate
Search cost	Exponential subset enumeration	Polynomial with n^2-pass cap
Approximate total time	$O(n \log n + nk + Ikn2^{2n/k})$ for balanced group sizes	$O(n \log n + n^4 k)$

Table 2: Comparison of the proposed heuristics.

4 Implementation and Input/Output

Both programs are written in C++17. They accept the input file path and k as command-line arguments. The input begins with the number of sets, followed by one whitespace-separated set of positive integers per line. Results are written to the `output/` directory under the name `<input_stem>_output.txt`.

For each set, the output records the original set and its sum, the k resulting groups and their sums, and the final maximum difference. The original set retains input order; elements within each output group are sorted in non-increasing order; and group lines are sorted lexicographically in non-increasing order by their element sequences.

5 Compilation and Execution

Both proposed algorithms are implemented in C++17 and can be compiled using a C++ compiler supporting the C++17 standard, such as GCC.

5.1 Compilation

To compile the energy-based local search implementation, execute:

```
g++ -std=c++17 -O2 k_partitions.cpp -o solution2
```

To compile the polynomial-time single-element go to the directory polynomial_solution and execute:

```
g++ -std=c++17 -O2 k_partitions.cpp -o solution1
```

The `-std=c++17` flag enables C++17 features, including the filesystem library, while `-O2` enables compiler optimizations.

5.2 Execution

Both programs accept two command-line arguments:

1. The path to the input text file.
2. The number of subsets k.

For example, on Linux or macOS:

```
./solution1 input/AaC_example_1.txt 4
./solution2 input/AaC_example_1.txt 4
```

On Windows PowerShell:

```
.\solution1.exe input\AaC_example_1.txt 4
.\solution2.exe input\AaC_example_1.txt 4
```

5.3 Input Format

The input file must have a `.txt` extension.

The first line specifies the number of input sets. Each subsequent line contains one set of positive integers separated by spaces.

For example:

```
2
25 20 18 15 12 10 8 7 5
8 7 6 5 4
```

Each set must contain at least k elements to ensure that all resulting subsets are non-empty.

5.4 Output Format

The programs automatically create an `output/` directory in the current working directory, if it does not already exist.

For an input file named `AaC_example_1.txt`, the resulting file is saved as:

```
output/AaC_example_1_output.txt
```

The output contains:

- The number of input sets and the value of k.
- The original elements and their total sum.
- The resulting k subsets and their individual sums.
- The final difference between the maximum and minimum subset sums.

Elements within each subset are sorted in non-increasing order, and the subsets are ordered lexicographically in non-increasing order.

Additionally, the programs print intermediate optimization results to the console, allowing the progression of the local-search algorithms to be observed.

6 Conclusion

Our team developed two heuristic approaches for balancing the sums of k nonempty groups. Both use the same efficient greedy initialization. The first searches a broad neighborhood of subset exchanges using a squared-energy objective and can produce improved partitions at exponential computational cost. The second restricts the neighborhood to single-element swaps, directly reduces the maximum sum difference, and uses a polynomial pass limit to guarantee polynomial runtime.